cem-fixed: Architecture Redesign

From Original Version to cem-fixed

Three Fatal Bugs and Four Engineering Fixes

Group Meeting Presentation

March 2026

Overview: What Was Wrong and What We Fixed

Defense Goal

Force same-class features into tight sub-clusters so the MIA attacker cannot reconstruct images.
Lower Inference SSIM = Better Privacy.

Original Version (Root Dir)

- × Loss direction **reversed** — pushes variance up
- × Variance **predicted by a network**, not computed geometrically
- × Slot Attention clusters **within one image**, not across samples

All three bugs together make CEM not only ineffective but potentially **harmful** to defense.

cem-fixed

- ✓ Loss direction corrected
- ✓ Geometric variance computation
- ✓ Cross-sample clustering via `FeatureMemoryBank`
- ✓ `proj_down` for tractable high-dim features
- ✓ Centroid `.detach()` to close gradient shortcut
- ✓ Deterministic inference

Module Architecture: Original vs cem-fixed

Original SlotCrossAttentionCEM

- z flat [B, feat_dim]
- `unsqueeze(1)` → [B, 1, D] — Bug 3
- SlotAttention (each image alone)
- CrossAttention (slots attend features)
- `head_logvar` (Linear+Softplus) — Bug 2
- CEM Loss **direction reversed** — Bug 1

cem-fixed SlotCrossAttentionCEM

- z flat [B, 2048]
- `proj_down`: Linear(2048→128) — Fix 4
- MemoryBank + batch concat per class — Fix 3
- SlotAttention [1, N_c, 128]
- Geometric Var $\sum \text{attn} \cdot (f - c^*)^2$ — Bug 2+3
- CEM Loss **direction correct** — Bug 1

Bug 1: Loss Direction is Completely Reversed

Original formula:

$$\mathcal{L} = \mathrm{ReLU}\Big(\log(\theta) - \log(\bar{v} + \varepsilon)\Big)$$

- Positive when $\bar{v} < \theta$ (variance **below** threshold)
- Penalizes **low variance** $\Rightarrow$ drives variance **up**
- Encoder pushed to **spread** same-class features
- Opposite to CEM intent — helps the attacker

cem-fixed formula:

$$\mathcal{L} = \mathrm{ReLU}\Big(\log(\bar{v} + \gamma) - \log(\theta)\Big)$$

- Positive when $\bar{v} > \theta$ (variance **above** threshold)
- Penalizes **high variance** $\Rightarrow$ drives variance **down**
- Encoder pushed to **compress** features into tight sub-clusters
- **Gradient direction matches CEM defense goal**

Threshold: $\theta = \texttt{var_thr} \times \texttt{reg_str}^2 + \gamma, \;\; \gamma{=}0.01$

Impact

With the original formula, increasing `lambd` would *actively degrade* privacy. Every step was working **against** the defense.

Bug 2: Variance Predicted by Network, Not Computed Geometrically

Original: neural network predicts variance

slots [B, S, D]

↓

`head_logvar` (Linear + Softplus)

↓

$\hat{v}$ = network output (free parameter)

- `head_logvar` has its own learnable weights
- Network can satisfy loss by **adjusting its own weights**, not the encoder
- CEM pressure is **absorbed by the prediction head**
- Encoder features are **unconstrained** — no defense effect

cem-fixed: geometric computation

all_feats $[1, N_c, D]$

↓

attn_norm (soft assignments)

↓

centroids = einsum(attn, feats).`detach()`

↓

$v = \sum \text{attn} \cdot (f - c^*)^2 \;/\; D$

- Variance = **actual Euclidean distance** from features to centroids
- No learnable parameters involved in variance computation
- Gradients go **directly to encoder features**

Bug 3: Slot Attention Clusters Within One Image, Not Across Samples

The largest semantic error. CEM needs cross-sample clustering; the original version does intra-sample operation.

Original: each image is processed alone

z_{flat} [B, D]

↓

`unsqueeze(1)` → [B, 1, D]

↓

SlotAttention(input=[B, 1, D])

8 slots compete over 1 token
⇒ all slots collapse to same point

cem-fixed: cluster same-class samples together

z_{proj} [B, 128] (all classes)

↓

select class c: batch_feats $[n_c, 128]$

↓

cat(batch_feats, bank_feats) $[N_c, 128]$

↓

SlotAttention(input=[1, N_c, 128])

8 slots compete over N_c samples of class c

Why it matters: CEM's goal is to find *sub-groups within each class*. This requires observing multiple samples of the same class simultaneously. With a single token as input, Slot Attention has nothing to cluster — 8 slots simply mirror the one input token.

Engineering Fix 1: FeatureMemoryBank

Problem: After fixing Bug 3, each class appears in a batch of 64 only ∼1–2 times (530 classes). $N_c < 2 \Rightarrow$ class **skipped entirely**.

FeatureMemoryBank design:

bank: Tensor[530, 64, 128]

ptr: LongTensor[530] (ring buffer ptr)

valid: LongTensor[530] (fill count/class)

`update(feats, labels) @no_grad`

`get_class_features(cls)` → [n, 128]

How it feeds SlotAttention:

1. Batch feats written to bank (`no_grad`)
2. Class c: read up to 64 historical feats
3. `cat([batch_feats, bank_feats])`
4. SA receives $N_c = n_{\text{batch}} + n_{\text{bank}} \gg 2$

Key properties:

- bank feats: **no gradient** (history)
- batch feats: **has gradient** (drives encoder)
- Non-`Module` ⇒ must override `.to(device)`

Engineering Fix 2: proj_down for High-Dimensional Features

Problem

FaceScrub encoder: $z = [B, 8, 16, 16] \xrightarrow{\text{flatten}} [B, 2048]$. SlotAttention directly on 2048-dim $\Rightarrow$ enormous parameter cost.

Without proj_down:

Layer	Size	Params
`to_q`	2048×2048	4.2M
`to_k`	2048×2048	4.2M
`to_v`	2048×2048	4.2M
GRU	2048×2048	16.8M
MLP	2048×4096	8.4M
Total		$>$ 100M

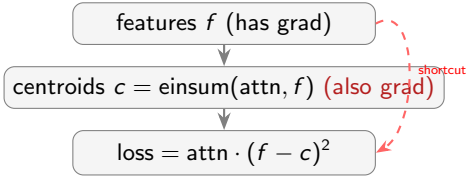
With proj_down (slot_dim=128):

Layer	Size	Params
`proj_down`	2048×128	0.26M
`to_q`	128×128	16K
`to_k`	128×128	16K
`to_v`	128×128	16K
GRU	128×128	66K
MLP	128×256	33K
Total		$\sim$0.5M

Gradient still flows: $\nabla_{\theta_f} = \frac{\partial \mathcal{L}}{\partial z_{\text{proj}}} \cdot \frac{\partial z_{\text{proj}}}{\partial z_{\text{flat}}} \cdot \frac{\partial z_{\text{flat}}}{\partial \theta_f}$. `proj_down` is differentiable — all 2048 dims receive CEM gradients.

Engineering Fix 3: Detach Centroids to Close Gradient Shortcut

The shortcut (without `.detach()`):

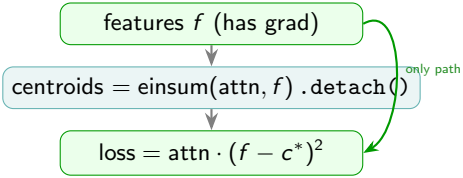


Two gradient paths:

1. features move toward centroid (correct)
2. centroid moves toward features (shortcut!)

Path 2 satisfies the loss with **zero encoder change**.

With `.detach()` (cem-fixed):



One gradient path only: Features must truly approach the fixed centroid. Encoder cannot "cheat" by moving the reference point.

Engineering Fix 4: Deterministic Inference

Original: always random slot initialization

learnable μ, σ

slots $= \mu + \sigma \cdot \varepsilon, \ \varepsilon \sim \mathcal{N}(0, 1)$ always

- During MIA evaluation, each forward pass gives **different attention distributions**
- CEM statistics fluctuate across evaluation runs
- Results are **not reproducible**; fair comparison is impossible

cem-fixed: train/eval split

learnable μ, σ

Training: $\mu + \sigma\varepsilon$ (stochastic)

Eval: μ (deterministic)

- Training: stochastic init preserves exploration, prevents slot collapse
- Inference: deterministic init gives **identical output** for same features
- MIA evaluation results are **fully reproducible**

Complete Bug & Fix Summary

	Issue	Original Behavior	cem-fixed
B1	Loss direction	$\mathrm{ReLU}(\log\theta - \log v)$: penalizes low var; **drives features apart**	$\mathrm{ReLU}(\log v - \log\theta)$: penalizes high var; drives features together
B2	Variance source	Network-predicted (`head_logvar`); pressure absorbed by head, **encoder unconstrained**	Geometric $\sum \mathrm{attn}(f - c^*)^2$; gradient direct to encoder
B3	Clustering object	Each image alone $[B, 1, D]$; 8 slots on 1 token = **meaningless**	Same-class samples $[1, N_c, D]$; cross-sample sub-groups
F1	Sample shortage	—	`FeatureMemoryBank`: 64 feats/class; every class has signal
F2	High-dim cost	SlotAttn in 2048-dim: 100M+ params	`proj_down`(2048→128): ∼0.5M, grad preserved
F3	Centroid shortcut	No detach: centroid chases features	`.detach()`: features must genuinely converge
F4	Non-determinism	Always stochastic slots	Deterministic eval; stochastic train

Why cem-fixed Should Outperform the Original Version

End-to-end signal path (cem-fixed):

Encoder $f(x) \to z$

↓

`proj_down` $\to z_{\text{proj}}$

↓

MemoryBank + class selection

↓

SlotAttention (cross-sample)

↓

Geometric variance per slot

↓

CEM Loss $\mathcal{L}_{\text{CEM}}$ (correct direction)

↓

$\nabla_{\theta_f} \mathcal{L}_{\text{CEM}} \to$ encoder update

What this achieves:

- CEM loss reflects **true geometric** feature distribution
- Gradient flows **unobstructed** to encoder via proj_down
- Every class gets meaningful signal every step (MemoryBank)
- SlotAttention finds **real sub-clusters** in feature space
- Encoder **genuinely forced** to compress same-class features

Defense Mechanism

Tight sub-clusters $\Rightarrow$ attacker cannot distinguish identities $\Rightarrow$ blurry reconstruction $\Rightarrow$ **lower Inference SSIM**.

Thank You

3 fatal bugs fixed + 4 engineering improvements
cem-fixed is ready to run on the server

20 experiments $\times$ $\sim$5h each on NV 5880 Ada (48GB)
Results in `run_exp.sh` $\rightarrow$ `log/summary.csv`